HELPTOA Bot

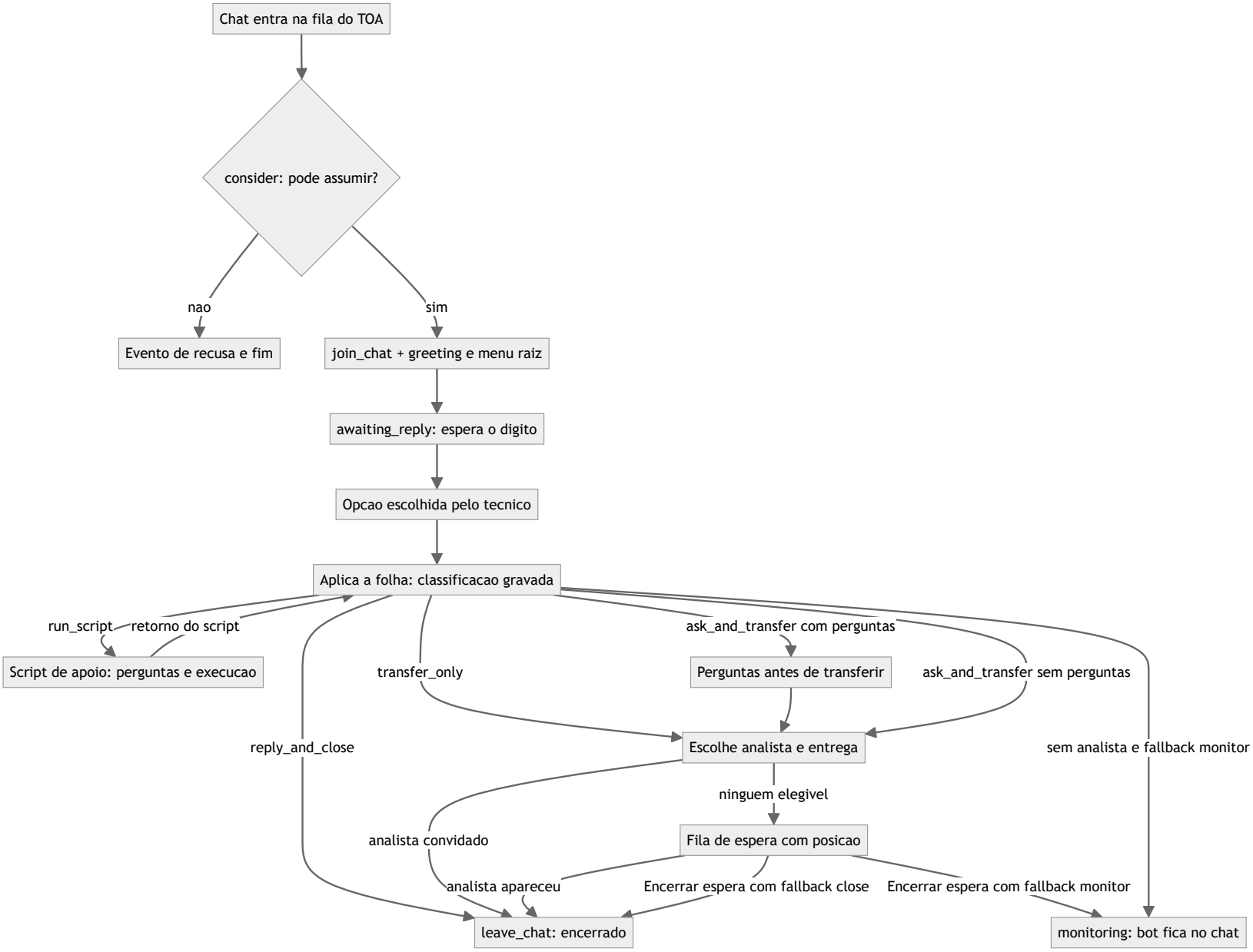
Fluxogramas do atendimento do chatbot — como está hoje

Retrato do código em 21/09/2026, conferido em `python/engine.py`, `python/routing.py`, `python/main.py`, `python/resume.py`, `python/form_open.py` e `schema.sql`. Visão detalhada: `docs/desenho-bot.md`.

1. Visão geral do atendimento
2. Filtros para assumir o chat
3. Menu, submenu, dígito, retry e timeout — inclui a **abertura form-aware** (3.1)
4. Folha escolhida: resposta, perguntas, script e transferência
5. Quem recebe: roteamento por caso
6. Entrega, fila de espera e fallback
7. Formulário de validação do TOA
8. Restart, resume e reciclagem do Chromium
9. Timers e saúde do processo
10. Status do chat e eventos registrados
11. Amarrações do painel

1. Visão geral do atendimento

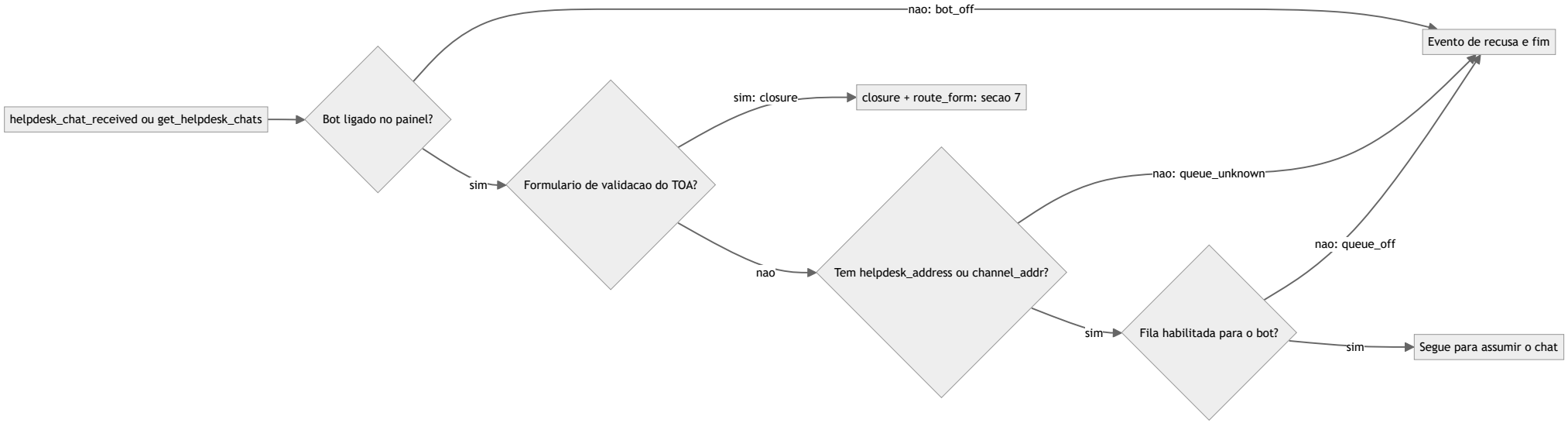
Caminho principal: o chat da fila habilitada é assumido, recebe o menu, é classificado e termina entregue a um analista, encerrado pelo bot ou em monitoramento.



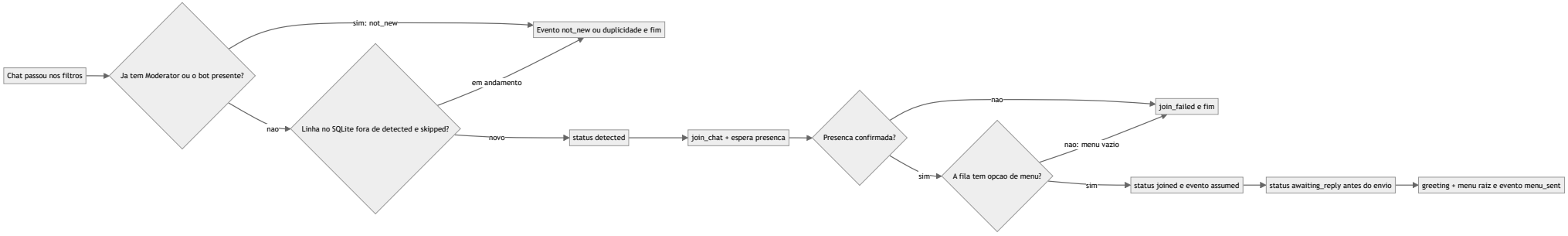
O bot é o único participante que fala pelo número da fila: entra com `join_chat`, grava o estado no SQLite **antes** de enviar cada mensagem e só sai com `leave_chat`. Nenhum texto de conversa vai para o banco — só status, classificação, respostas de coletas e timestamps.

2. Filtros para assumir o chat

Tudo isso vive em `Engine.consider`. Cada recusa gera evento próprio; nada é silencioso. Primeiro os filtros de entrada:



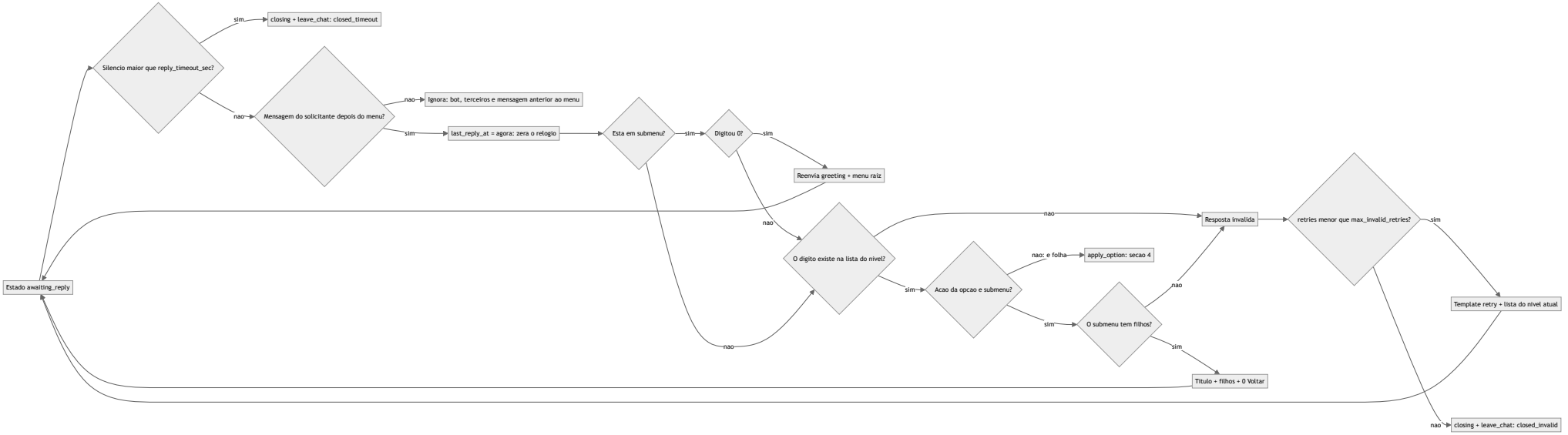
Depois a assunção: o bot só entra quando o chat ainda é novo e a presença é confirmada.



A espera de presença leva até 10 s (40 tentativas de 0,25 s) e, se o ACK do `join_chat` não confirmar, o bot confere em `get_active_chats`. O mesmo chat nunca reinicia a conversa: com linha em `detected` ou `skipped` o bot ignora a duplicidade; em um ciclo novo o `moved_at` é zerado.

3. Menu, submenu, dígito, retry e timeout

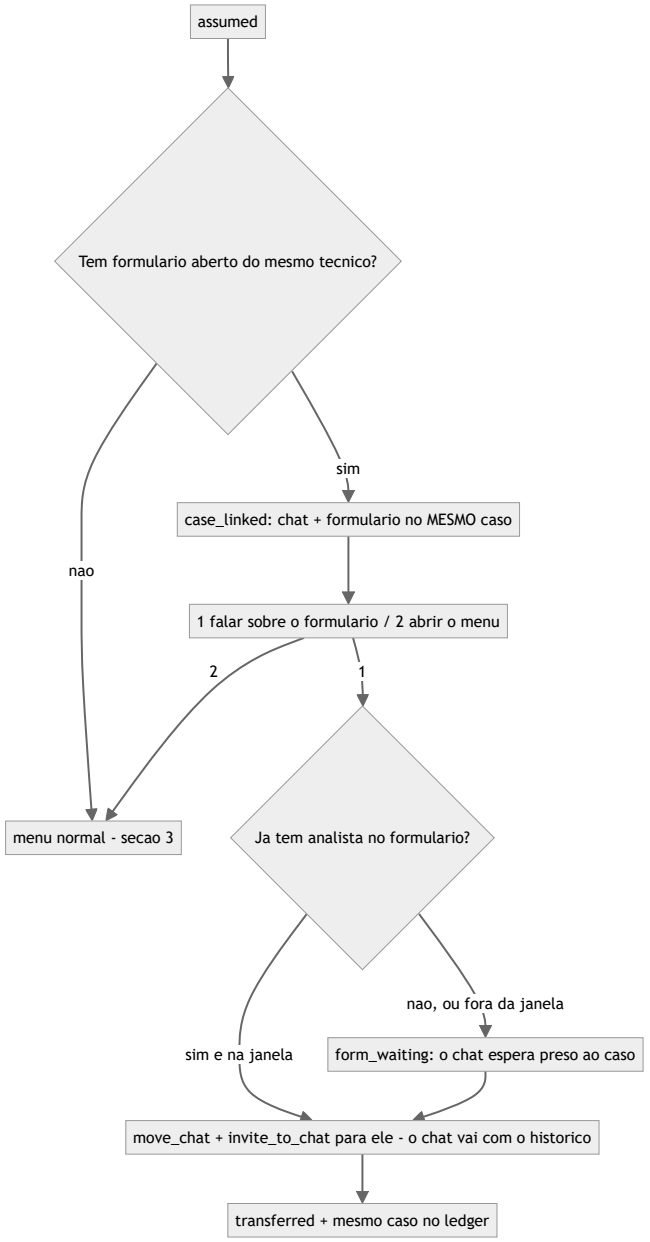
O estado é gravado antes do envio (`awaiting_reply` + `last_prompt_at`), a opção é escolhida pelo `sort_order` igual ao dígito e o relógio mede o **silêncio do solicitante**, não o tempo desde a pergunta.



São consideradas só mensagens do técnico que abriu o chat (`started_addr`), depois do `menu_sent_at` e com texto. Resposta inválida **também** reinicia o relógio: `closed_timeout` só acontece com silêncio real. O menu nunca volta sozinho para a raiz — `0` só funciona dentro de um submenu. Ao voltar de restart, `replay_pending_reply` processa a resposta que chegou com o bot fora (evento `reply_replayed`) antes de qualquer timeout.

3.1 Abertura form-aware: técnico com formulário em andamento

Quando o chat assumido é de um técnico que **já tem formulário de validação aberto**, o bot não manda o menu: ele oferece **1) Falar sobre o formulário** / **2) Abrir o menu**. Chat e formulário passam a ser **um caso só** (mesmo `case_key` no ledger): encaminhado um, o outro vai junto.

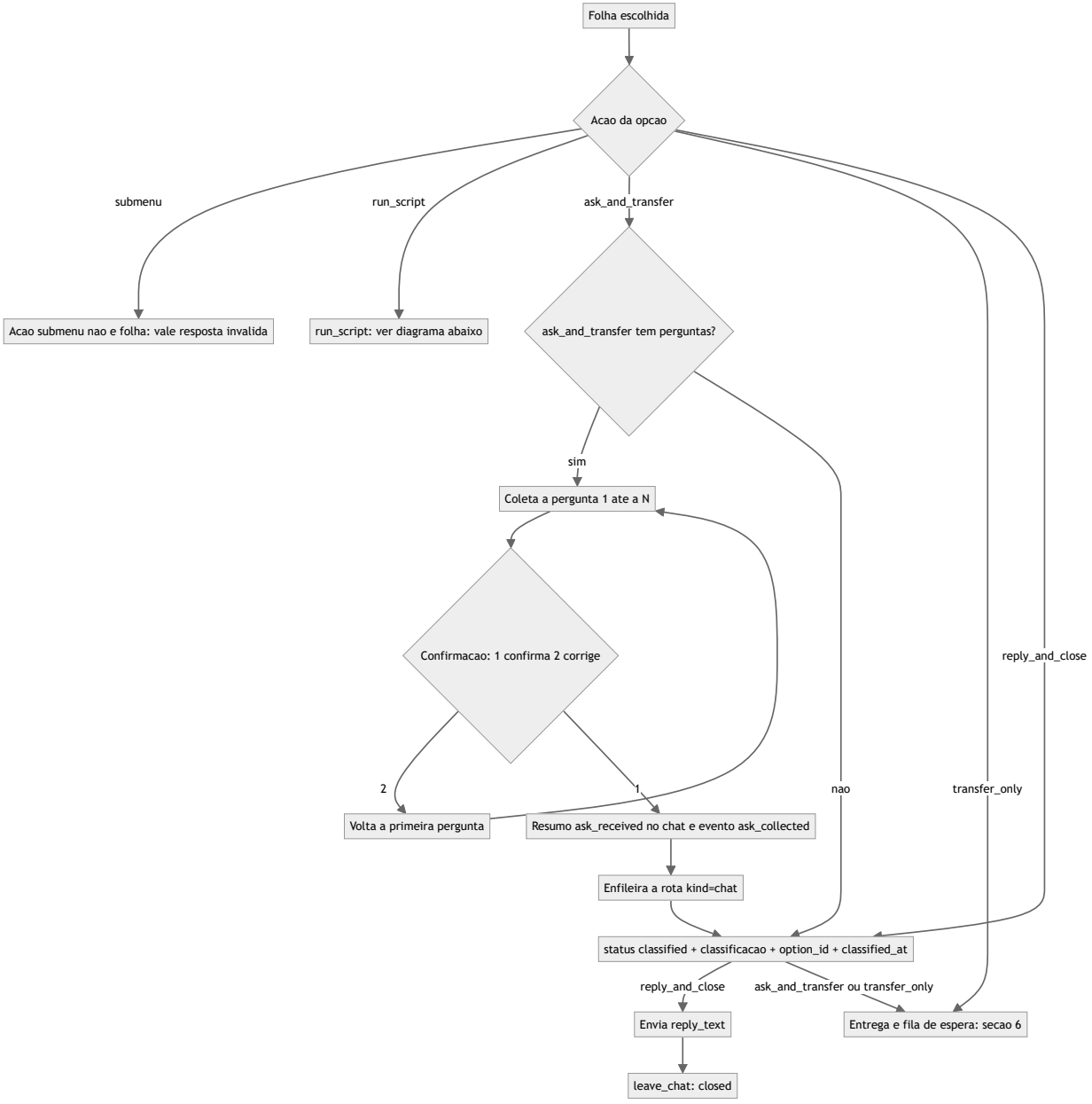


Correlação (qual formulário é do técnico): por `address`; por nome quando `affinity_match_name` = true; e, como o formulário do TOA normalmente **não traz o address do técnico** (o ledger chega a gravar `case_key='n:message scenario agent'`), o fallback é o **formulário aberto mais recente da mesma fila** que ainda não está vinculado a outro chat — o evento `case_linked` registra a regra usada. **Estados do formulário** (como na extensão do TOA): *pendente*, *em atendimento* (o analista entrou) e *passado* (o analista saiu / atendimento concluído) — o bot lê isso das **mensagens do formulário** (`get_chat_messages`, só leitura), no boot, na abertura e numa varredura de 1 por tick; passado sai de cena

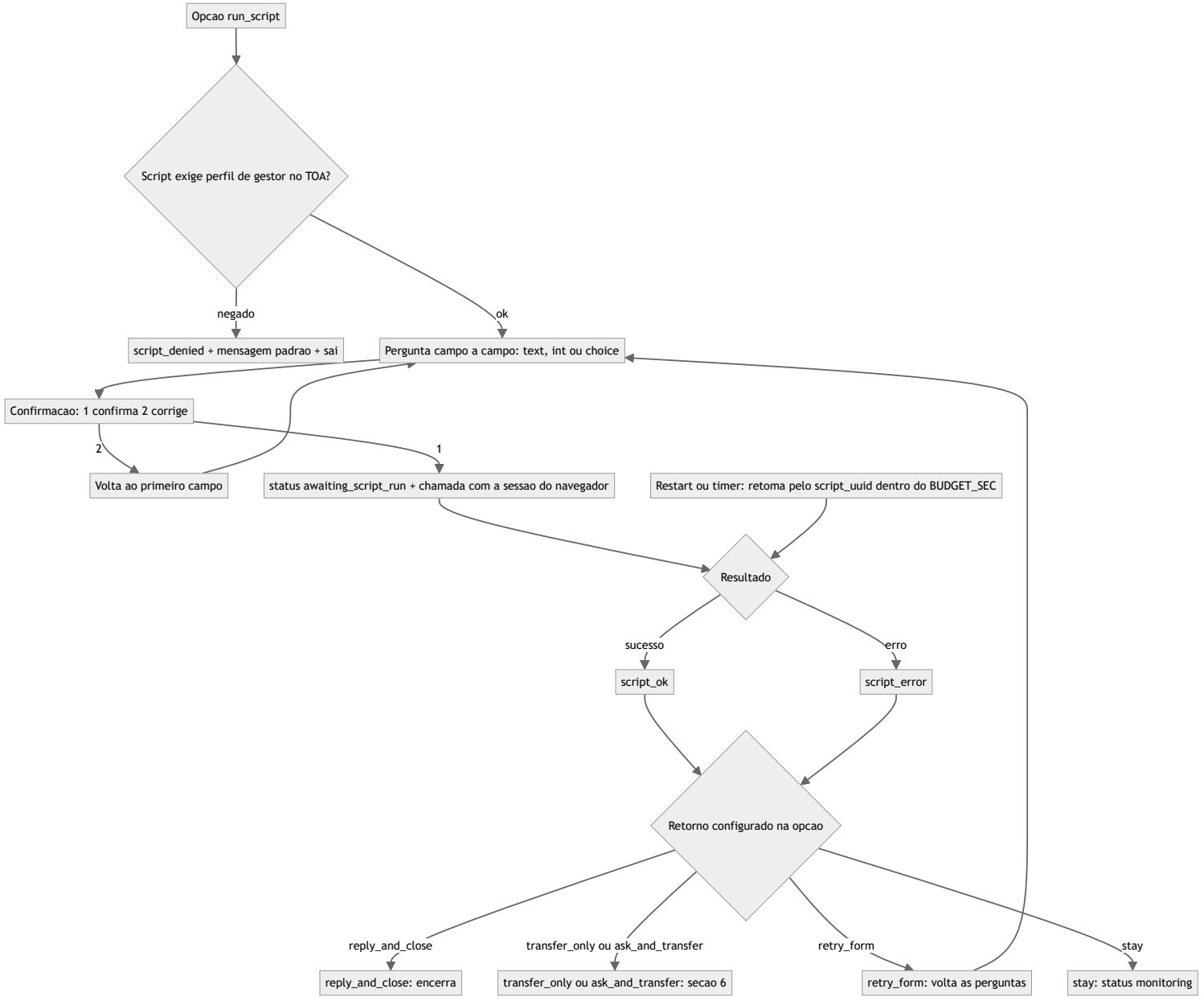
(`form_closed`) e nada é decidido se o TOA não responder. O vínculo é **definitivo**: formulário já oferecido a um chat não é oferecido de novo. **Índice do formulário:** fica acompanhado até um fim explícito, `chat_removed`, TTL de 12 h ou caso já encerrado no ledger; aviso de *fila* (`helpdesk_chat_removed`, que o `move_chat` para o analista gera) **não** tira o formulário, e o boot reconstrói o índice do ledger antes do resume (a varredura de boot não fecha caso de formulário). Eventos de diagnóstico: `form_indexed`, `form_not_open`, `form_dropped`, `form_ignored_payload`, `form_queue_left`, `form_forget`, `form_state`, `form_closed`, `form_aware_skip`. O chat do ramo 1 entra na fila com `expects_owner=1`: espera o analista do formulário **sem gastar tentativa nem cair no fallback**, e a entrega das duas peças juntas acontece quando uma delas é atribuída (`case_sibling_delivered`). Textos: `form_pending`, `form_pending_wait` e `form_waiting` (aba Templates).

4. Folha escolhida: resposta, perguntas, script e transferência

A folha grava a classificação e resolve a ação. `ask_and_transfer` com perguntas cadastradas coleta no chat antes de tentar entregar; `run_script` chama o script de apoio com a sessão do navegador.



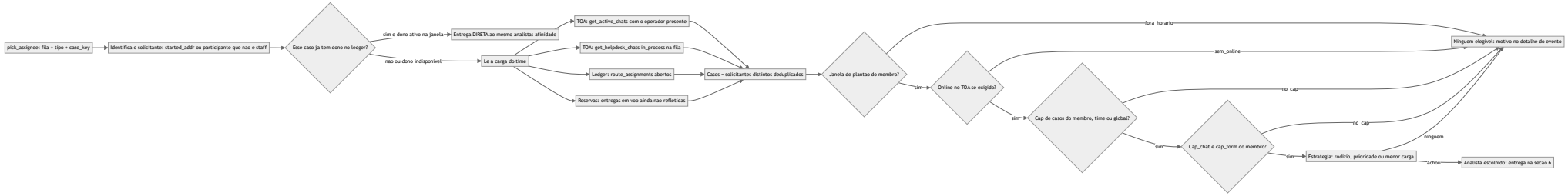
Ordem das mensagens do wizard: o resumo (`ask_received`) sai **sem** o `ask_transferring`; com analista elegível o bot manda `ask_transferring` e entrega, sem analista quem fala é o aviso de posição na fila. O progresso fica no banco (`ask_option_id`, `script_field_index`, `script_answers` em JSON `q1..qN`), então sobrevive ao restart.



Status do script: `awaiting_script_answer` → `awaiting_script_confirm` → `awaiting_script_run`. Campo inválido reusa a mesma validação do wizard de perguntas e conta em `retries`; esgotou, `closed_invalid`. O `tick_timeouts` retoma script preso em `awaiting_script_run`.

5. Quem recebe: roteamento por caso

Um único motor para chat e formulário, com a mesma fonte de carga. A unidade é o **caso**: um solicitante distinto. Formulário + chat do mesmo técnico contam como um caso só.



Ponto da decisão	Regra hoje
Afinidade	Caso com dono no ledger e dono ativo + na janela recebe direto, mesmo com o cap cheio. Online só barra se `affinity_require_online` estiver ligado. É por isso que o chat de quem já tem formulário vai para o mesmo analista.
Carga por caso	Soma as fontes deduplicando por `chat_id` e por solicitante. Sem resposta do TOA a carga **não zera**: valem ledger e reservas, e o evento registra `carga_toa=indisponivel`.
Janela de plantão	`route_start`/`route_end` do membro; vazio ou início igual ao fim = 24 h; madrugada 22:00-06:00 atravessa a meia-noite. Lida no fuso America/Sao_Paulo.
Limites	`cap_cases` (membro → time → `max_cases_per_agent`, padrão 4) e os sub-limites `cap_chat` / `cap_form`, que contam só o próprio tipo.
Estratégia	`round_robin` = próximo depois do último escolhido; `priority` = maior peso, empate no rodízio; `least_load` = menor número de casos, empate por peso e rodízio. O rodízio é independente por tipo.
Auditoria	O detalhe do evento mostra `time`, `estrategia`, `cap`, `membros`, `janela`, `fora_horario`, `sem_online`, `so_chat`, `no_cap`, `casos`, `carga` e `tentativas` — dá para saber por que cada um foi descartado.

6. Entrega, fila de espera e fallback

Com analista escolhido, a entrega é `move_chat` (uma única vez por chat) + `invite_to_chat`. Sem ninguém elegível, o chat entra na fila de espera com posição e fica lá até alguém aparecer, o técnico sair ou alguém usar *Encerrar espera*.

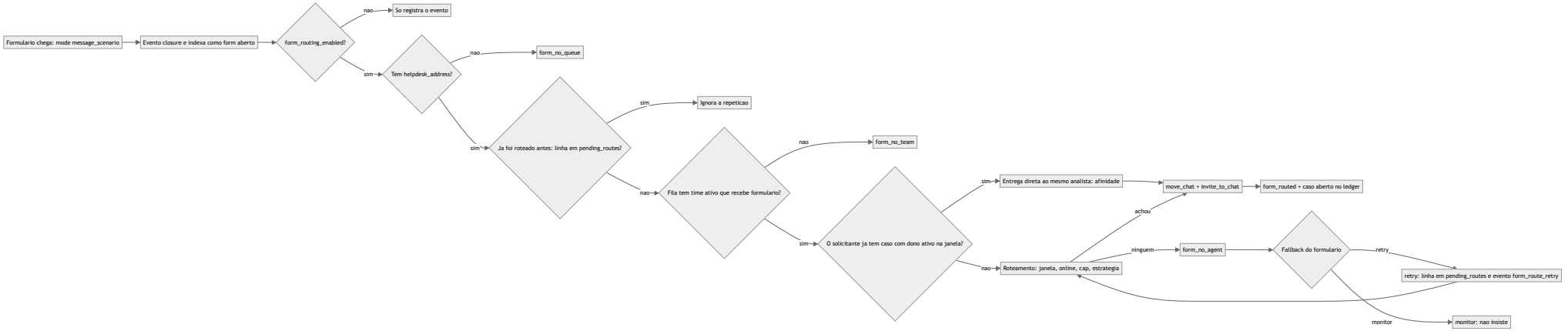


A fila de espera vive na `pending_routes` (`kind=chat` **ou** `kind=form`): a ordem de entrega é FIFO **por time, contando chat e formulário na mesma linha** — mas o **número falado** conta só chats (o técnico não vê os formulários à frente dele) e só aparece a partir de `queue_top_pos` (padrão 5); acima disso ele recebe o texto de progresso `queue_progress` ("Você avançou na fila"). Entrar na fila emite `queued` para os dois tipos (não é falha), e o **formulário espera em silêncio** (sem mensagem). **Nenhum** `move_chat` antes de existir analista — por isso o técnico não vê "estou transferindo" seguido de "você é o Nº 1 da fila". A espera **vence** em `queue_max_wait_sec` (`0` = sem limite): ao vencer, o bot registra `queue_expired`, avisa o técnico (texto `queue_expired` quando o fallback é monitorar) e aplica o fallback do time; para o formulário a rota é encerrada (`form_no_agent`) em vez de girar para sempre. O botão *Encerrar espera* antecipa o mesmo caminho. Com o chat **já movido** e o convite recusado, as tentativas contam normalmente (senão ele ficaria alternando para sempre). Nada disso se perde em restart: a rota está no banco.

Desfecho	Status do chat	Evento
Analista convidado	`closed`	`transferred`
Convite recusado após o move	segue na rota	`transferred_partial` + `route_retry`
Entrou na fila de espera	`classified`	`queued` / `queue_position`
Atribuído depois de esperar	`closed`	`queued_assigned` (com `espera=NNs`)
Espera encerrada (vencimento ou painel)	`closed_no_agent` ou `monitoring`	`queue_expired` + fallback do time
Sem analista e sem fila	`closed_no_agent` ou `monitoring`	`no_agent`
Técnico saiu / chat removido	`closed_originator_left` / `left`	`left` e rota fechada como `failed`

7. Formulário de validação do TOA

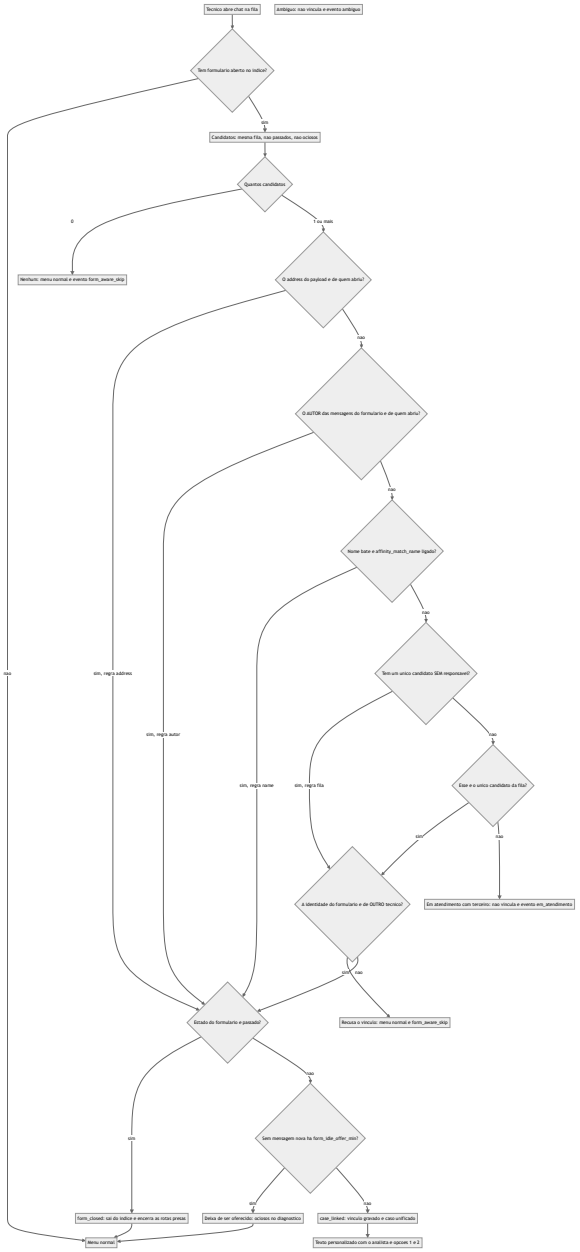
O formulário não é chat de menu: o bot **não entra**, não envia mensagem e não encerra. Ele só roteia, com a mesma dinâmica do chat e por tentativas agendadas em segundo plano.



Uma fila desabilitada para chat **continua** roteando formulário: o que manda é o `form_routing_enabled` (global) e o campo "recebe formulário" do time. O mesmo formulário nunca é roteado duas vezes — a marca fica na `pending_routes`, inclusive após restart — e o formulário **espera em silêncio**: o aviso de posição só existe no chat. Quando o formulário do técnico já tem um caso aberto, o chat dele vai para o mesmo analista (afinidade), contando como um caso só.

7.1 Identidade do caso: chat e formulário do mesmo técnico

Um caso é o **solicitante**. O chat e o formulário do mesmo técnico precisam cair no mesmo caso — e nunca no caso de outra pessoa. A identidade sai, nesta ordem, do `address` do payload, do **autor das mensagens do formulário** (o técnico escreve lá), do nome (quando ligado) e, só com **um único candidato livre**, da fila.



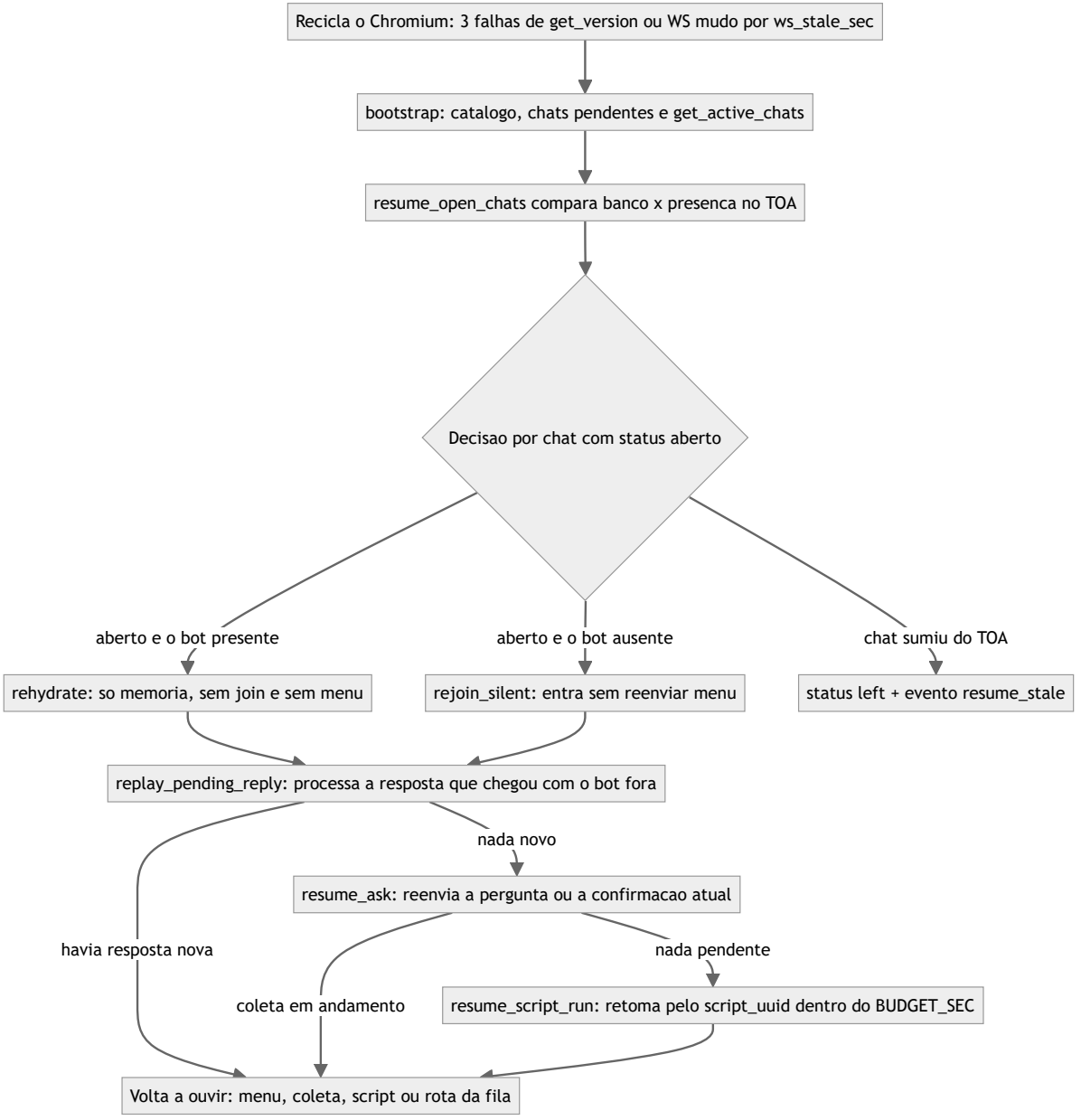
O palpite por fila só vale com **um** candidato **sem responsável** e sem conflito de identidade: era o vínculo cego por recência que movia o formulário de outro técnico para este caso (o caso "trocava de analista" no painel). Formulário que **já está com um analista do time** só entra por identidade — e, quando é o único da fila, continua sendo oferecido (é o técnico falando do próprio formulário). Leitura **forçada** do estado antes de responder: a mensagem ao técnico nunca sai de cache, formulário encerrado não é oferecido de novo e formulário **ocioso** (sem mensagem nova por `form_idle_offer_min`, sem analista dentro) deixa de ser oferecido.



O formulário que **já está com um analista** não é movido de novo: o TOA recusa com `e=10` e o bot adotava o retry infinito (mesmo analista atrás de `move_chat` a cada minuto). Agora ele adota o dono que as mensagens mostram, conta a carga do analista e encerra a rota; se a recusa persistir sem dono identificado, a rota para no teto (`form_move_refused`) em vez de girar para sempre. E o formulário **sem sinal de vida** (o TOA não avisa a saída do analista) sai de cena pela janela de vida: deixa de ser oferecido e o caso fecha — `form_ocioso` (120 min sem analista dentro) ou `form_ocioso_analista` (240 min **com** analista dentro, o estado real dos casos que travavam os caps). O índice também é podado quando o caso no ledger já encerrou de verdade (`form_forget motivo=caso_encerrado`).

8. Restart, resume e reciclagem do Chromium

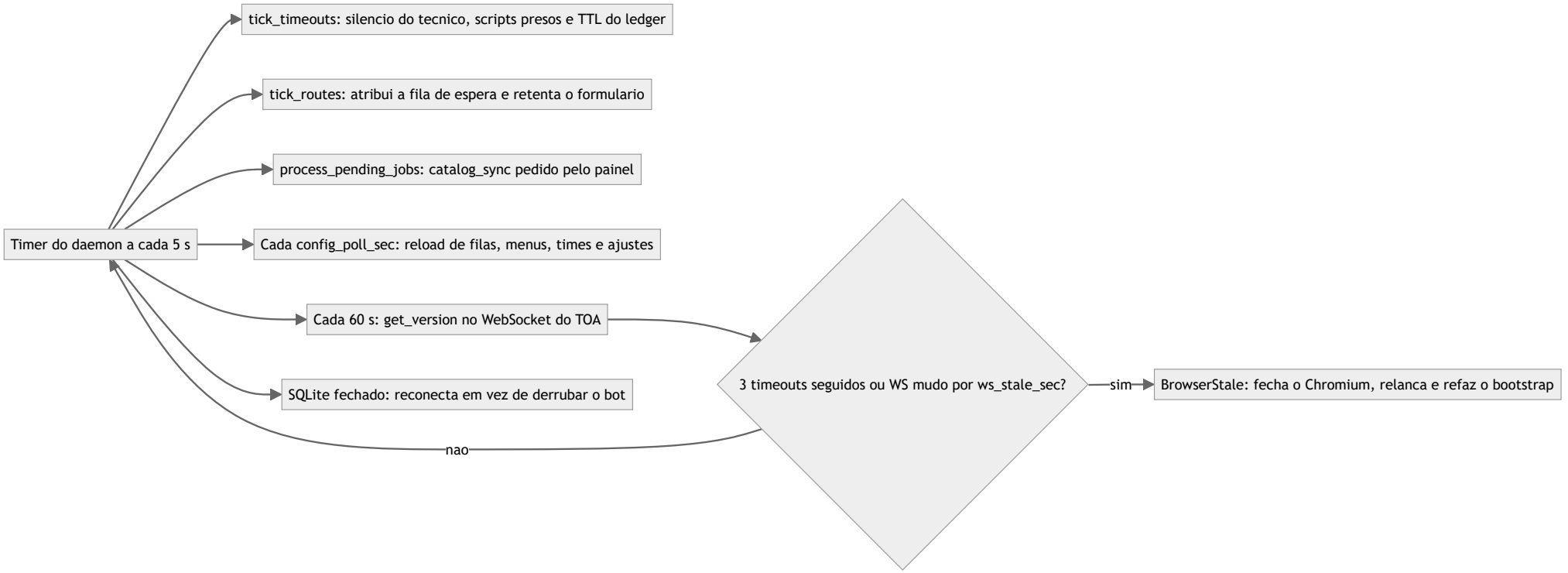
O SQLite guarda status, `menu_node_id`, `retries`, `last_prompt_at`, a coleta e as rotas. A memória RAM some; depois do bootstrap o bot retoma em vez de tratar o chat como novo.



Status retomados: `joined`, `menu_sent`, `awaiting_reply`, `classified`, `monitoring`, `awaiting_script_answer`, `awaiting_script_confirm` e `awaiting_script_run`. O menu nunca é reenviado no resume e `reply_replayed` acontece antes de qualquer timeout — sem isso a resposta perdida viraria `closed_timeout` cinco minutos depois. A fila de retry do formulário, a fila de espera do chat e o ledger também sobrevivem. `SIGTERM` fecha o Chromium e sai com código 0.

9. Timers e saúde do processo

Um único laço de 5 s empurra o atendimento; o WebSocket só recebe e entrega eventos. Se o TOA ou o navegador travam, o bot recicla o Chromium e refaz o bootstrap — sem perder o que está no banco.



Ajuste (aba Ajustes / Roteamento)	Padrão	Para que serve
`reply_timeout_sec`	300 s	Silêncio do solicitante antes de encerrar; qualquer resposta (mesmo inválida) reinicia o relógio.
`max_invalid_retries`	3	Respostas inválidas no menu, no wizard e nas perguntas do script antes de `closed_invalid`.
`config_poll_sec`	15 s	Intervalo de recarga das configurações do painel.
`ws_stale_sec`	180 s	WebSocket mudo que dispara a reciclagem do Chromium.
`route_assign_ttl_sec`	43200 s	Caso órfão no ledger sem evento de saída é fechado e a vaga volta.
`route_reserve_ttl_sec`	300 s	Validade da reserva de carga feita na hora da entrega.

Ajuste (aba Ajustes / Roteamento)	Padrão	Para que serve
`queue_wait_enabled`	ligado	Fila de espera do chat com posição e sem limite de tempo; desligado volta a `transfer_attempts`.
`form_route_attempts` / `form_route_retry_interval_sec`	—	Tentativas e intervalo do formulário em segundo plano.

10. Status do chat e eventos registrados

Uma linha por `chat_id` em `chats`; cada passo vira evento em `bot_events` (sem texto de conversa).

Status	Significado
`detected`	Chat identificado, ainda sem presença confirmada.
`joined`	Bot entrou no chat.
`menu_sent`	Menu enviado (o estado já é `awaiting_reply`).
`awaiting_reply`	Esperando o dígito, com timeout por silêncio.
`awaiting_form_choice`	Abertura form-aware: esperando **1** (falar sobre o formulário) ou **2** (abrir o menu); o timeout por silêncio continua valendo como salvaguarda.
`awaiting_script_answer`	Wizard de perguntas ou script: preenchendo um campo.
`awaiting_script_confirm`	Wizard/script: confirmando os dados (1 confirma, 2 corrige).
`awaiting_script_run`	Script em execução com a sessão do navegador.
`classified`	Folha aplicada, com classificação gravada — inclui quem está na fila de espera.
`monitoring`	Bot fica no chat sem menu, aguardando o desfecho.
`closed`	Encerrado pelo bot depois de transferir ou responder.
`closed_invalid`	Encerrado por respostas inválidas.
`closed_timeout`	Encerrado por silêncio real do técnico (resposta perdida no restart é recuperada em `reply_replayed`).
`closed_no_agent`	Sem analista elegível e fallback do time = encerrar.
`closed_originator_left` / `left`	O técnico saiu / o chat foi removido ou o bot saiu.

Grupo	Eventos
Entrada e assunção	`assumed`, `menu_sent`, `not_new`, `queue_off`, `queue_unknown`, `bot_off`, `join_failed`
Conversa e desfecho	`ignored_reply`, `reply_replayed`, `send_failed`, `classified`, `transferred`, `no_agent`, `closed_invalid`, `closed_timeout`, `closed_no_agent`, `left`, `resume_stale`

Grupo	Eventos
Perguntas antes de transferir	`ask_collected`, `route_retry`, `transferred_partial`
Fila de espera	`queued`, `queue_position`, `queued_assigned`, `queue_expired`
Scripts	`script_ok`, `script_error`, `script_denied`
Formulário de validação	`closure`, `form_routed`, `form_no_agent`, `form_route_retry`, `form_no_team`, `form_no_queue`
Abertura form-aware	`case_linked` (chat + formulário no mesmo caso), `form_aware_greeting`, `form_aware_choice`, `form_aware_wait`, `form_aware_skip`, `case_sibling_delivered`
Índice do formulário	`form_indexed`, `form_not_open`, `form_dropped`, `form_ignored_payload`, `form_queue_left` (aviso de fila; o formulário segue aberto), `form_forget`, `form_state` (pendente/em atendimento/passado), `form_closed` (virou passado), `case_key_divergente` (identidade do payload venceu a chave da rota)
Resposta ignorada	`message_ignored` — mensagem do técnico em status que o bot não trata (antes era silencioso)
Catálogo	`catalog_synced`, `sync_error`

O que **ainda não existe**: uso de `transfer_targets` / `menu_option_targets` (código morto — decidir entre implementar ou tirar da UI), o `CHAT_TYPE_ROUTING` nativo do TOA, a validação em produção do escopo do `get_active_chats` e os campos `action_if_form`/`reply_text_if_form` do menu (legado sem uso: o painel grava vazio e o motor ignora — a abertura form-aware vive no vínculo do caso, seção 3.1).

11. Amarrações do painel

O que o painel liga e o efeito imediato no atendimento. O motor lê só as tabelas de catálogo, menu, times e roteamento — RBAC é assunto do painel.

Aba / campo	Liga	Efeito no atendimento
Filas	`enabled` e sincronização do catálogo	Fila desligada: o bot não assume o chat (`queue_off`), mas ainda roteia o formulário dela.
Menus ↔ filas	1 fila = 1 menu	Sem opção cadastrada o chat cai em `join_failed` (menu vazio).
Opção submenu	raiz → filhos	Um nível por vez; `0` volta para a raiz.
Opção folha	classificação + ação + texto	Encerrar, transferir, perguntar antes de transferir ou rodar script.
Perguntas da opção	1 a 20 perguntas (`text`, `int`, `choice`)	Coleta no chat antes de tentar entregar; sem perguntas o comportamento é o antigo.
Templates	`greeting`, `menu`, `retry`, `closing`, `ask_received`, `ask_transferring`, `queue_position`	Textos das mensagens, inclusive `{pos}` da fila de espera.
Times	membros, Ordem, janela de plantão e filas do time	Define quem pode receber e em que ordem; membro inativo não recebe.
Roteamento: globais	bot ligado, formulário roteado, estratégia, caps, tentativas, fallback, fila de espera	Formulário desligado: só registra `closure`. Bot desligado: nenhum chat é assumido.
Roteamento: por time	estratégia, caps, online, plantão, tentativas, fallback, recebe formulário	Campo vazio herda o global (membro → time → global).
Roteamento: por membro	peso, cap chat, cap form, casos, recebe formulário	Peso só vale na estratégia por prioridade.
Roteamento: fila de espera	posição, tempo de espera, tentativas e *Encerrar espera*	O botão vence a espera na hora e deixa o motor aplicar o fallback do time.
Dashboard	classificações por categoria, resumo do formulário e fila de espera do chat	Leitura do que está acontecendo agora.
Atividade	`bot_events` com filtro por tipo	É onde se audita cada decisão de roteamento, com o detalhe dos descartes.
Chats	status, classificação, retries e respostas das coletas	Sem conteúdo de conversa.
Usuários / Grupos (super-only)	contas, grupos, permissões e escopo de dados	Vale só para o painel: negar por padrão, sem marcação e sem `scope.all_*` o grupo não vê dado nenhum.

Resumo em uma frase: o bot assume o chat da fila habilitada, oferece o menu, classifica a folha escolhida (com perguntas ou script quando configurado) e então decide quem recebe — contando **casos por solicitante** no TOA, no ledger e nas entregas em voo, com **afinidade** para o formulário do mesmo técnico, **fila de espera com posição e sem limite de tempo** no chat, timeout medido por **silêncio real do solicitante** com recuperação da resposta que chegou com o bot fora, e retry assíncrono para o formulário, que nunca entra na conversa nem é encerrado pelo bot.